# Deep Learning

05-06-26

## Adagrad: Adaptive gradient descent

In SGD with momentum:

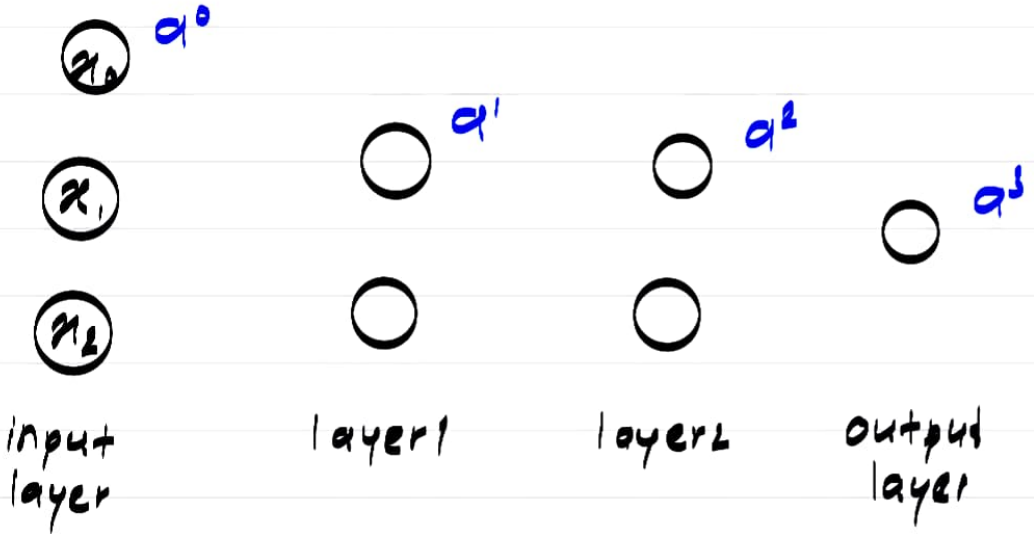
$$v_t = \gamma v_{t-1} + \alpha g_t \qquad g_t = \partial L / \partial w_t$$

$$w_{t+1} = w_t - v_t$$

- here $\gamma$ and $\alpha$ are same for every weight
- Adagrad proposes different learning rate for all weights

- Adagrad solved exploding and vanishing gradient problem.

## Exploding and Vanishing gradient problem:



$z^1 = w^1 x = w^1 a^0$

$a^1 = f_1(z^1)$

$z^2 = w^2 a^1$

$a^2 = f_2(z^2)$

$z^3 = w^3 a^2$

$a^3 = f_3(z^3)$

$Loss = loss\ function\ (Y - \hat{Y}) = loss\ function\ (Y - a^3)$

$$\frac{\partial L}{\partial w^3} = \frac{\partial L}{\partial a^3} \cdot \frac{\partial a^3}{\partial z^3} \cdot \frac{\partial z^3}{\partial w^3}$$

$$= \frac{\partial L}{\partial a^3} \cdot \frac{\partial f_3(z^3)}{\partial z^3} \cdot \frac{\partial z^3}{\partial w^3}$$

$\downarrow$

$f_3'(z^3) \rightarrow$ derivative of activation function

$$\frac{\partial L}{w^2} = \frac{\partial L}{\partial a^3} \cdot \frac{\partial a^3}{\partial z^3} \cdot \frac{\partial z^3}{\partial a^2} \cdot \frac{\partial a^2}{\partial z^2} \cdot \frac{\partial z^2}{\partial w^2}$$

$$= \frac{\partial L}{\partial a^3} \cdot \frac{\partial f_3(z^3)}{\partial z^3} \cdot \frac{\partial z^3}{\partial a^2} \cdot \frac{\partial f_2(z^2)}{\partial z^2} \cdot \frac{\partial z^2}{\partial w^2}$$

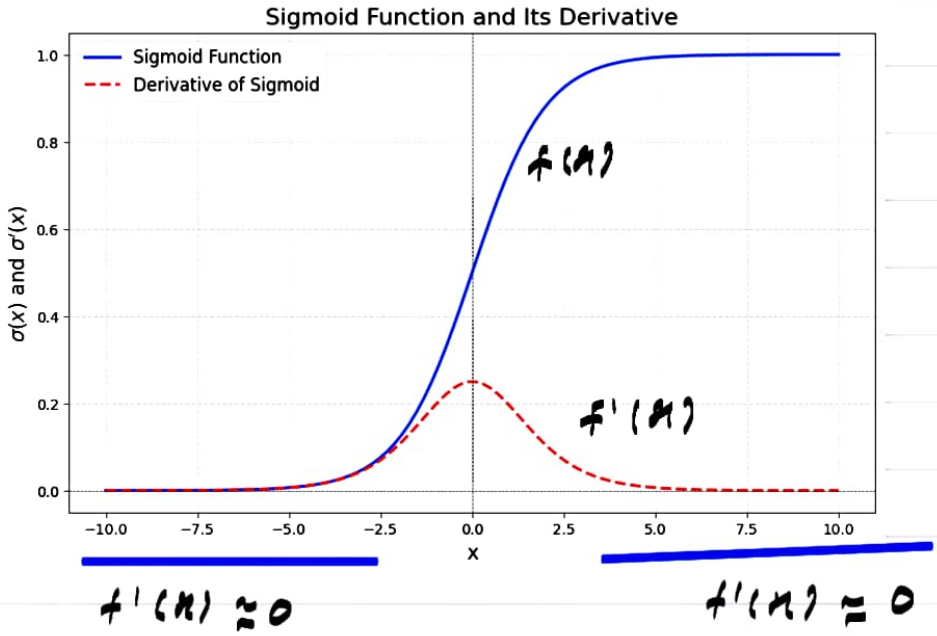
$\downarrow$ $\qquad$ $\downarrow$

$f_3'(z^3) \qquad f_2'(z^2)$

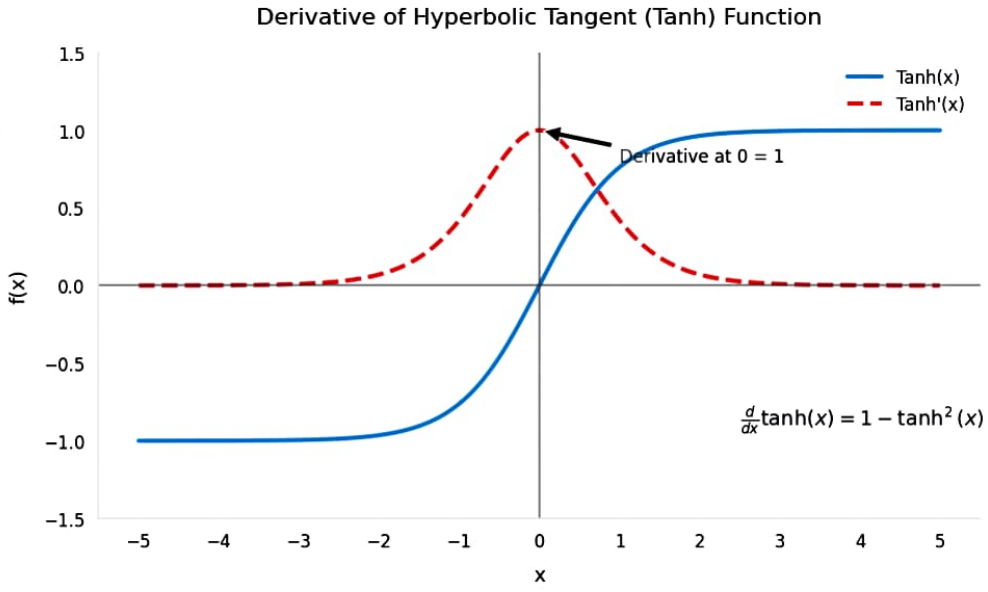
if these two values become very small $\approx 0$ then eventually $\frac{\partial L}{\partial w^2}$ will become very small

$$w^2_{new} = w^2_{old} - \alpha \, \partial L / \partial w^2_{old}$$

as $\partial L / \partial w^2_{old}$ is very small, weight updation won't be significant.

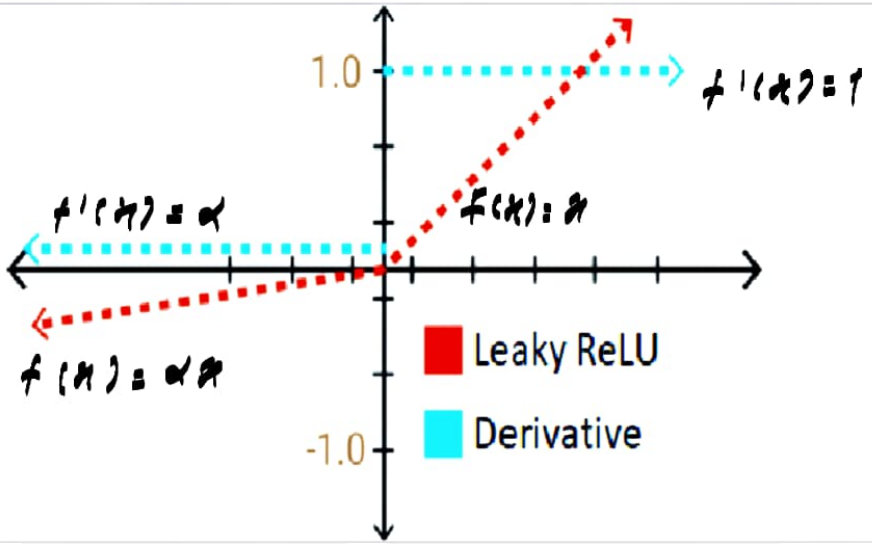


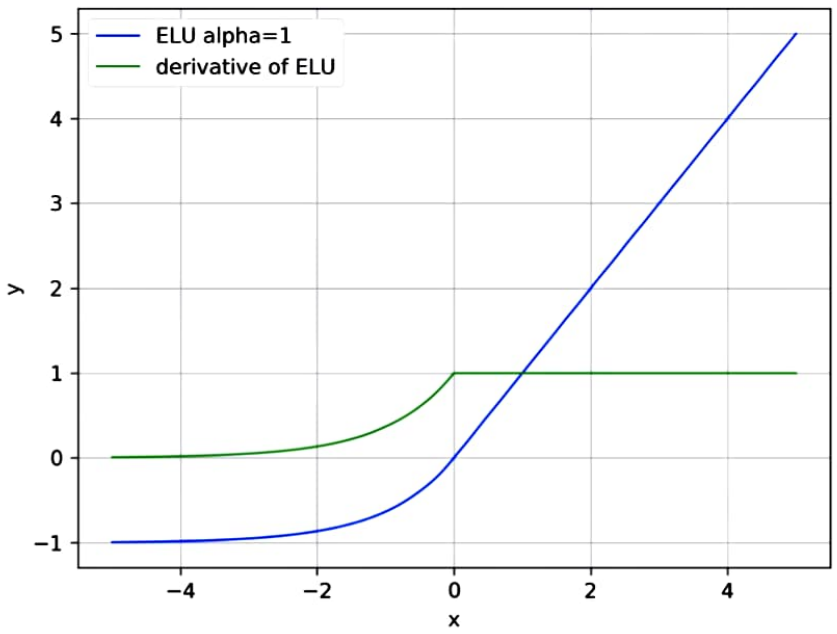

When gradients of activation functions become very small, then weight updation after every step is not significant, this makes model training very slow. This is vanishing gradient issue.



## Solutions for vanishing gradient problem:

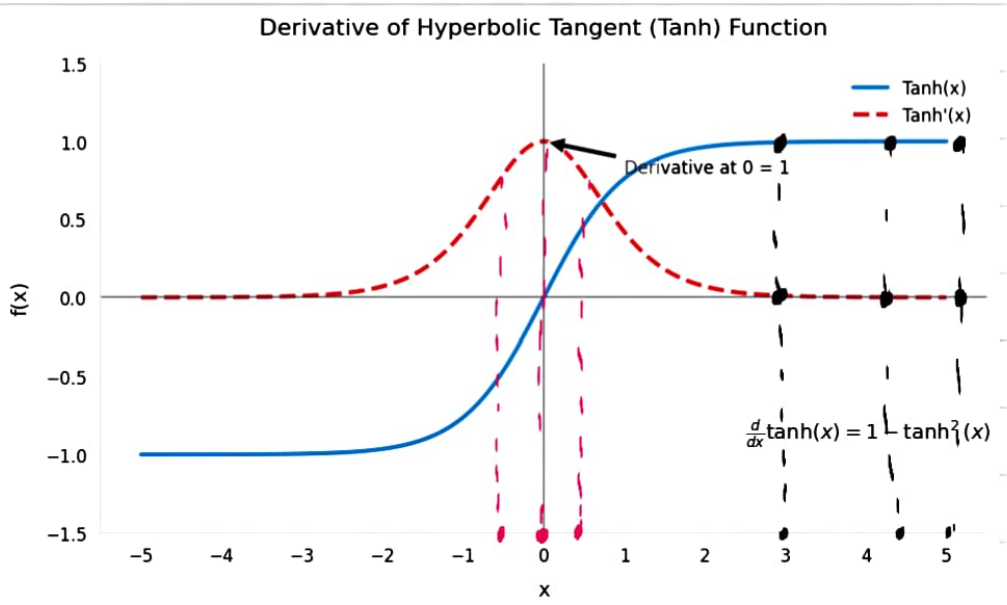
- Normalization of inputs at every layer
- use relu based activation functions
  - Leaky relu, ELU, etc
- reduce model complexity by reducing number of hidden layers
- use advanced optimizers such as - Adagrad, Adam...




there are very less regions in relu based activations, where gradients could become zero - this reduces chances of vanishing gradient issue.

# How normalizing inputs to every layer helps in reducing vanishing gradients?




lets say we've used tanh act func in layer 2

$$z^2 = w^2 a^1$$

$$a^2 = f_2(z^2)$$

$a^1$ is input to layer 2

there are 3 nodes in layer 1, $a^1$ will have 3 values

$a^1 = [10, 20, 30]$

after multiplying with $w^2$

$z^2 = [3, 4.5, 5]$ $\quad a^2 = f_2(z^2) = [1, 1, 1]$

$f_2'(z^2) = [0, 0, 0]$

↓

vanishing gradient problem

if we normalize $a^1$

$a^1 = [10, 20, 30]$

$\mu$ - mean

$\sigma$ - std dev

$$\frac{a^1 - \mu}{\sigma}$$

normalized $a^1 = [-1.2, 0, 1.2]$ → zero centered and closer to origin

after multiplying with $w^2$

$z^2 = w^2 a^1 = [-0.4, 0, 0.3]$

$a^2 = f_2(z^2) = [-0.7, 0, 0.6]$

$f_2'(z^2) = [-0.8, 1, 0.75]$

↓

no vanishing gradient issue.

Exploding gradient issue:

$$\frac{\partial L}{\partial w^1} = \frac{\partial L}{\partial a^3} \frac{\partial a^3}{\partial z^3} \cdot \frac{\partial z^3}{\partial a^2} \cdot \frac{\partial a^2}{\partial z^2} \cdot \frac{\partial z^2}{\partial a^1} \cdot \frac{\partial a^1}{\partial z^1} \cdot \frac{\partial z^1}{\partial w^1}$$

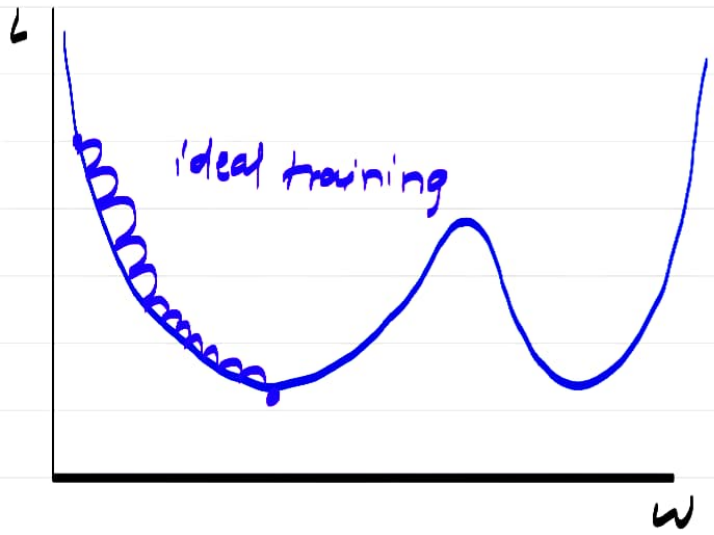
$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

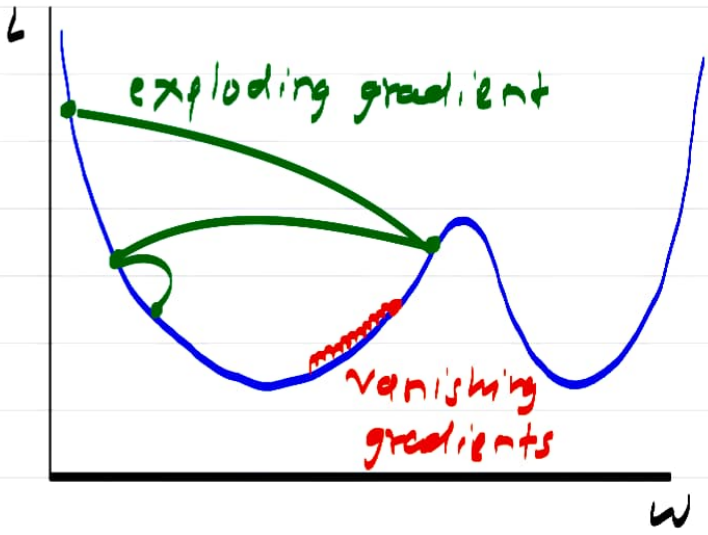
$$f_3'(z^3) \quad w^3 \quad f_2'(z^2) \quad w^2 \quad f_1'(z^1) \quad a^0$$

if the multiplication of these values return a very large gradient value for $\partial L / \partial w^1$ - it will cause exploding gradient issue.

Exploding gradient issue - too large gradients

noisy weight updations




In Adagrad optimizer :

$$\alpha_t = \alpha_{t-1} + (g_t)^2 \qquad g_t = \partial L / \partial w_t$$

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$ } adaptive learning rate

$\eta$ - fixed +ve constant / initial learning rate

$\varepsilon$ - small +ve value

$$w_{t+1} = w_t - \eta_t g_t$$

$$w_{t+1} = w_t - \eta_t \frac{\partial L}{\partial w_t}$$

$$w_{t+1} = w_t - \eta_t \frac{\partial L}{\partial w_t}$$

$$= w_t - \frac{\eta}{\sqrt{\alpha_t + \varepsilon}} g_t$$

vanishing gradient → $g_t$ ↓ small value

$$\alpha_t = \alpha_{t-1} + (g_t)^2$$

$\alpha_t$ ↓ small value

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$ if $\alpha_t$ ↓ $\eta_t$ ↑↑

$$w_{t+1} = w_t - \eta_t g_t$$

↓

$\eta_t \times g_t$ → not too small

not too large

↓

remove vanishing gradient issue.

Exploding gradient → $g_t$ ↑ very large value

$$\alpha_t = \alpha_{t-1} + (g_t)^2$$

$\alpha_t$ ↑ very large value

$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$ ↓ very small value

$w_{t+1} = w_t - \eta_t g_t$ → not too large